

Kendall's notation and an introduction to queueing theory

440-2216/01 Kvalita služeb • Lecture 02

Jan Rozhon, Miroslav Vozňák • Department of Telecommunications, FEECS

Where we are going

Part 1

Queueing systems

Arrivals, servers, waiting positions, and Kendall's A/B/C/D/E/F notation for classifying a queue

↔ Exercise 02, Steps 1–2

Parts 2–3

From Poisson arrivals to M/M/1

The Poisson process, the exponential distribution, Little's law, and the M/M/1 delay and occupancy formulas

↔ Exercise 02, Preparation and Steps 3–4

Part 4

Traffic engineering

The erlang, offered traffic, and the classic Erlang B, Erlang C and Engset formulas for dimensioning lines

↔ README Question 1

One question runs through the whole lecture: a shared resource serves requests one at a time — **how long does a request wait, and how many lines does the resource need?**

What you should be able to do

1. Read Kendall's A/B/C/D/E/F notation and state the assumptions behind M/M/1.
2. Explain why the exponential distribution is memoryless and why that makes Poisson-driven queues tractable.
3. Compute the offered traffic, utilization, mean occupancy, and mean delay of an M/M/1 system.
4. Apply Little's law to relate occupancy and delay for any stable queueing system.
5. Compute a blocking probability with Erlang B and a waiting probability with Erlang C.

Keep asking: **which system, which rate, which distribution, and does it have a steady state?**

A queue is what happens when arrivals outrun service, even briefly

QoS dimensions the servers, the buffer, and the discipline so that delay and loss stay bounded

Part 1

Queueing systems and Kendall's notation

A familiar queue, first

Picture a small coffee shop with two baristas and a line by the door.

- Customers **arrive** at some average rate — some minutes two show up, some minutes none.
- Each **barista** takes a while to make a drink, then is free for the next customer.
- If both baristas are busy, new customers **wait in line**.
- The shop decides **who is served next** — usually whoever arrived first.

Coffee shop

Queueing system

Customers arriving

Arrival rate λ

One barista's pace

Service rate μ

Number of baristas

Number of servers

The line by the door

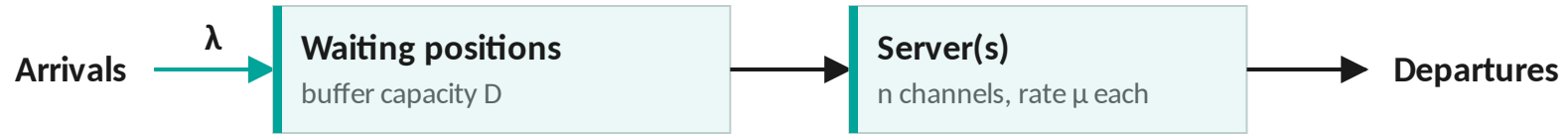
Waiting positions

"Next, please" order

Queueing discipline

Every formula in this lecture is this picture, made precise enough to compute with.

Anatomy of a queueing system



Queueing discipline orders the waiting positions: FIFO/FCFS, LIFO/LCFS, SIRO, priority, ...

Kendall notation A/B/C/D/E/F packages all of these choices into one string.

Same shape as the coffee shop — now with the vocabulary we will use for the rest of the lecture. A queueing system is described by five things:

- **Arrival rate** $\lambda [s^{-1}]$ — requests offered per second
- **Service rate** $\mu [s^{-1}]$ — requests one server completes per second
- **Number of servers** — how many requests can be served at once
- **Number of waiting positions** — buffer size; default is unbounded
- **Queueing discipline** — FIFO/FCFS, LIFO/LCFS, SIRO (serve in random order), priority queueing (PQ), ...

What we want to know

The customer's side

- Queueing delay and total delay in the system
- Number of requests waiting, or in the system
- The probability of having to wait, or of being blocked outright

The provider's side

- Server utilization / occupancy
- Buffer utilization / occupancy
- Revenue obtained versus revenue lost to blocking
- Customer satisfaction — grade of service (GoS)

The rest of this lecture builds the formulas that connect these quantities to λ , μ , and the number of servers.

Kendall's notation — A/B/C/D/E/F

- **A** — inter-arrival time distribution
- **B** — service time distribution
- **C** — number of servers
- **D** — system capacity K (waiting + in service). Default: infinite
- **E** — population size (number of potential sources). Default: infinite
- **F** — queueing discipline. Default: FIFO

Distribution codes for A and B:

- **M** — exponential (*Markovian* / memoryless)
- **D** — deterministic (constant, "degenerate")
- **E_k** — Erlang of order k (sum of k exponential stages)
- **G** — general (any distribution)

Trailing fields may be omitted when they take their default value: A/B/C means A/B/C/ ∞ / ∞ /FIFO.

Kendall's notation — worked examples

$M/M/1 = M/M/1/\infty/\infty/\text{FIFO}$

- Poisson arrivals
- exponentially distributed service times
- one server, unbounded queue, FIFO

The model this lecture builds toward.

$M/M/N/N = \text{Erlang B}$

- Poisson arrivals, exponential service
- N servers, **no waiting positions**
- a blocked call is lost, not queued

The model behind Part 4's first formula.

$M/D/1/20$ — Poisson arrivals, **constant** service time, one server, room for 20 requests total (19 waiting while one is served).

Checkpoint — reading Kendall's notation

A support hotline is modeled as G/M/2/5. What does each symbol say about how it works, and what happens to caller number six?

Predict first · discuss with a neighbour · then reveal

General inter-arrival times, exponential (Markovian) call-handling times, 2 agents, room for 5 callers total.

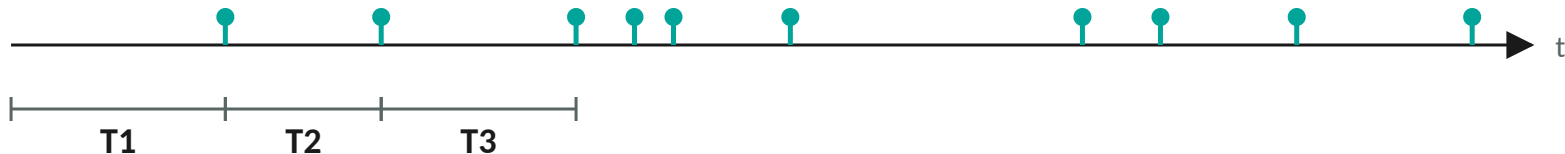
Two callers are served at once; up to three more wait on hold. A sixth caller, arriving while all five places are occupied, is **blocked** — Kendall's notation says nothing about what happens to a blocked request; that is a system-design choice (busy tone, overflow route, voicemail, ...).

Part 2

The Poisson process and the exponential distribution

The Poisson process — arrivals at a constant, memoryless rate

ARRIVALS ON THE TIME AXIS



Inter-arrival times $T_1, T_2, \dots$ are i.i.d. $\text{Exponential}(\lambda)$ — the description used in simulation.

Equivalently: the number of arrivals in any interval of length t is Poisson with mean λt .

A **homogeneous Poisson process** models arrivals happening independently, at a constant average rate λ , completely at random.

Two equivalent descriptions

Counting arrivals

The number of arrivals N in an interval of length t is Poisson-distributed:

$$P(N = k) = \frac{(\lambda t)^k}{k!} e^{-\lambda t}, \quad k = 0, 1, 2, \dots$$

Mean $E(N) = \lambda t$, variance $D(N) = \lambda t$.

Timing arrivals

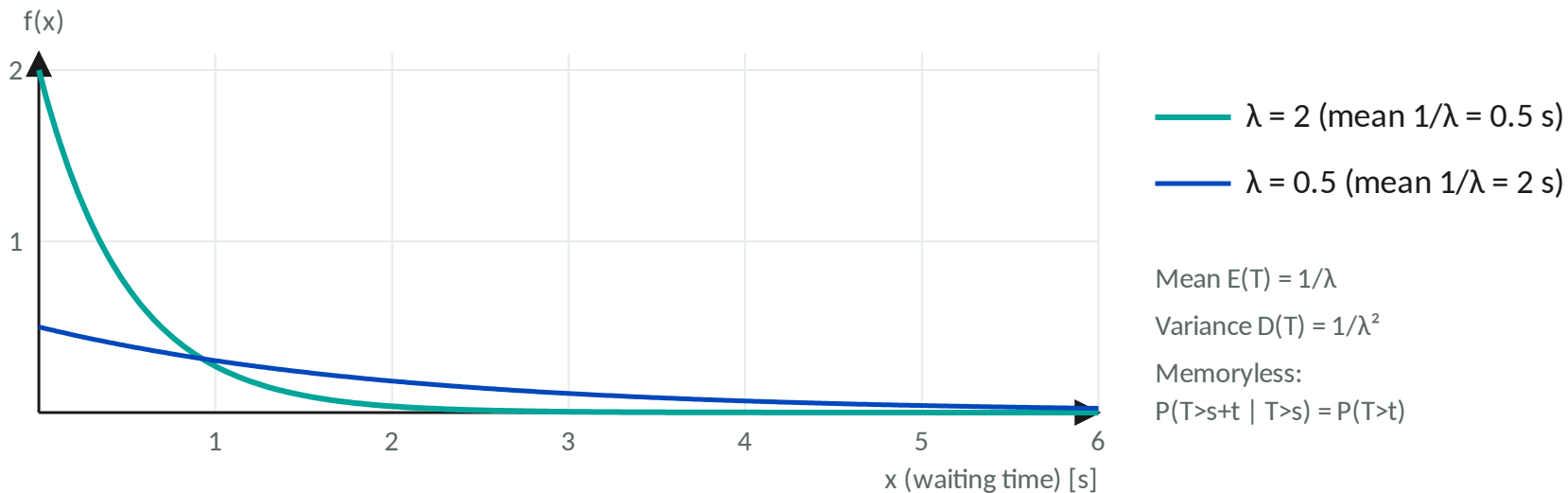
The **inter-arrival times** are independent and exponentially distributed with rate λ :

$$f(x) = \lambda e^{-\lambda x}, \quad x \geq 0$$

Mean $E(T) = 1/\lambda$, variance $D(T) = 1/\lambda^2$.

These describe the same process. A packet source that draws each inter-arrival time from $\text{Exponential}(\lambda)$ generates a Poisson process with rate λ — this is how the simulation builds arrivals.

The exponential distribution is memoryless



$$P(T > s + t \mid T > s) = P(T > t)$$

The time still to wait does not depend on how long you have already waited. A server that has been busy for ten minutes is, under this model, no more likely to finish in the next second than one that just started.

Checkpoint — why not measure elapsed service time?

A router port is mid-way through transmitting a large packet when a new one arrives. Under the M/M/1 model, does knowing that the current packet has already been "in service" for a while change your prediction of how much longer it needs?

Predict first · discuss with a neighbour · then reveal

No — that is exactly what memorylessness means for the exponential service-time model.

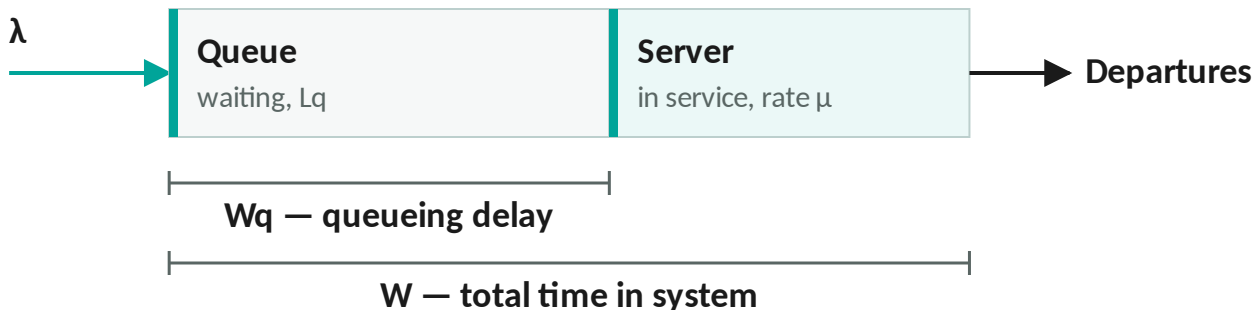
The remaining service time still has the same $\text{Exponential}(\mu)$ distribution, regardless of elapsed time. This is a modeling assumption, not a law of physics: real transmission times for a fixed-size packet are close to **deterministic**, which is why M/D/1 and G/G/1 models also matter in practice.

Part 3

Little's law and the M/M/1 queue

Little's law

L (system occupancy) = L_q (queue occupancy) + ρ (the fraction in service)



$$L = \lambda W, \quad L_q = \lambda W_q$$

L — mean number **in the system** [-]

λ — mean **admitted** rate [s^{-1}]

W — mean time **in the system** [s]

Holds for **any** stable queueing system, whatever the arrival and service distributions are — a very general accounting identity, not specific to M/M/1.

Little's law — a worked example

Three independent message classes share a system with **no waiting queue** — each message is served immediately (ample capacity), so W is simply the mean handling time.

Class	Rate	Mean time W
1	120 msg/min	200 ms
2	20 msg/min	10 s (6 msg/min served)
3	10 msg/min	30 s

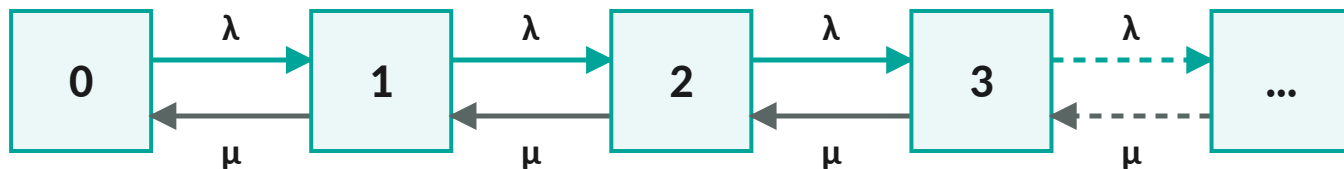
$$L_1 = \frac{120}{60} \cdot 0.2 = 0.4$$

$$L_2 = \frac{20}{60} \cdot 10 = 3.33$$

$$L_3 = \frac{10}{60} \cdot 30 = 5.0$$

$$L = L_1 + L_2 + L_3 = 8.73$$

The M/M/1 queue



State = number of customers in the system. Arrivals (λ) push it up; services (μ) pull it down.

M/M/1: Poisson arrivals (λ), exponential service times (μ), one server, unbounded queue, FIFO.

$$\rho = \frac{\lambda}{\mu} \quad (\text{utilization} = \text{offered traffic, in erlang})$$

A steady state exists only for $\rho < 1$. At or above this threshold, the mean queue length grows without bound.

Where the formulas come from

At steady state, the flow **up** across any cut of the chain must equal the flow **down** across it — the rate of leaving state n upward equals the rate of arriving there from above:

$$\lambda p_n = \mu p_{n+1} \quad \Rightarrow \quad p_n = \rho^n (1 - \rho)$$

p_n is the steady-state probability of finding n requests in the system — a geometric distribution in n . Summing $n p_n$ over all n is exactly where

$$L = \sum_{n=0}^{\infty} n p_n = \frac{\rho}{1 - \rho}$$

comes from — the formula on the next slide is this sum, not a rule to memorize independently of the picture.

M/M/1 — occupancy and delay

In the system:

$$L = \frac{\rho}{1 - \rho}$$

$$W = \frac{1}{\mu - \lambda}$$

In the queue (waiting only):

$$L_q = \frac{\rho^2}{1 - \rho}$$

$$W_q = \frac{\rho}{\mu - \lambda}$$

L, L_q are dimensionless [-]; W, W_q are times [s]. All four grow without bound as $\rho \rightarrow 1$:

ρ	0.5	0.8	0.9	0.95	0.99
$L = \rho/(1 - \rho)$	1	4	9	19	99
W , as a multiple of $1/\mu$	2×	5×	10×	20×	100×

This is why links are run well below full utilization. With k identical servers (M/M/ k), utilization is $\rho = A/k$ instead; these four formulas are M/M/1 only.

M/M/1 — from packet size to delay

Poisson arrivals at $\lambda = 5$ packet/s, exponential sizes with mean $\bar{b} = 100$ B, link rate $R = 8\,000$ bit/s.

$$S = \frac{8\bar{b}}{R} = \frac{8 \cdot 100}{8000} = 0.1 \text{ s} \Rightarrow \mu = 10 \text{ s}^{-1}$$

$$\rho = \frac{\lambda}{\mu} = 0.5$$

$$L = 1, W = 0.2 \text{ s}$$

$$W_q = 0.1 \text{ s waiting, } 1/\mu = 0.1 \text{ s transmitting}$$

This is Exercise 02, Step 4: a `SwitchPort` with exponential inter-arrivals and exponential packet sizes approximates M/M/1 when its buffer is large enough that loss is negligible.

Checkpoint — delay near saturation

A link runs at $\rho = 0.9$. The server is busy only 90% of the time — it is not saturated. Why does the mean delay W already feel enormous?

Predict first • discuss with a neighbour • then reveal

Because $W = 1/(\mu - \lambda)$ has ρ in its denominator through $1 - \rho$, not in a bounded ratio.

At $\rho = 0.9$, W is already $10\times$ the mean service time $1/\mu$; at $\rho = 0.99$ it is $100\times$. The server being idle 10% of the time is enough for a long queue to build up during the busy stretches and only slowly drain — occasional idleness does not undo an accumulated backlog.

Part 4

Traffic engineering: the erlang and the Erlang formulas

Traffic engineering

Erlang B

Blocking probability of a **loss system**: no waiting positions, a blocked call is refused.
M/M/N/N.

Erlang C

Probability that a call must **wait**: blocked calls stay in the system until served. M/M/N.

Engset

Like Erlang B, but for a **finite population** of sources — low-traffic scenarios where Erlang B's infinite-population assumption breaks down.

Agner Krarup Erlang (1878–1929), a Danish mathematician, founded traffic engineering and queueing theory at the Copenhagen Telephone Company.

The erlang — a unit of traffic load

1 erlang (Erl) = full-time occupancy of one channel: 60 minutes of traffic carried in 60 minutes, or equivalently one call continuously in progress.

For $N_x(t)$ simultaneously busy channels, the **carried traffic** over a period T is

$$Y = \frac{1}{T} \int_{t_0}^{t_0+T} N_x dt \quad [\text{Erl}]$$

Offered traffic A [Erl] is what arrives; **carried traffic** Y [Erl] is what gets served; **lost traffic** Z [Erl] is what is blocked and routed elsewhere or dropped:

$$Y = A - Z$$

For an M/M/1 queue with no loss, $A = \rho$ — the same offered traffic from Part 3, just in telephony units.

Traffic engineering vocabulary

- **BHT** — Busy Hour Traffic: offered/carried traffic during the network's busiest hour
- **BHCA** — Busy Hour Call Attempts
- **BHCC** — Busy Hour Call Completions
- **CPS** — Calls Per Second
- **AHT** — Average Hold Time (mean service/holding time, $1/\mu$)

Networks are dimensioned for the **busy hour**, not the daily average — a link sized for average load would be overloaded every peak.

Erlang B — the loss formula

$$E_B(A, N) = \frac{\frac{A^N}{N!}}{\sum_{i=0}^N \frac{A^i}{i!}}$$

A — offered traffic [Erl] · N — number of channels · E_B — probability an arriving call is **blocked**

Assumes Poisson arrivals, an **infinite population** of sources, and **no waiting positions** (M/M/N/N). erlang.com/calculator/erlb

Erlang B — worked example

$N = 3$ trunks, offered traffic $A = 1.5$ Erl:

$$\frac{A^i}{i!} : \quad 1, 1.5, 1.125, 0.5625 \quad (i = 0..3)$$

$$E_B = \frac{0.5625}{1 + 1.5 + 1.125 + 0.5625} = \frac{0.5625}{4.1875}$$

$$E_B \approx 13.4\%$$

about one call in seven is blocked

Adding a 4th trunk (A still 1.5 Erl) drops E_B to about 4.2% —
diminishing blocking, not diminishing traffic.

Checkpoint — what if blocked calls didn't hang up?

Erlang B assumes a blocked call is simply lost. What would have to change about the system for a blocked call to **wait** instead — and what happens to the formula's denominator once waiting is allowed?

Predict first · discuss with a neighbour · then reveal

The system needs somewhere to put a waiting call — unlimited waiting positions instead of none.

That single change turns $M/M/N/N$ (Erlang B) into $M/M/N$ (Erlang C): callers beyond the N busy servers are held, not dropped. The next two slides build exactly that formula.

Erlang C — the waiting-time formula

$$E_C(A, N) = \frac{\frac{A^N}{N!} \cdot \frac{N}{N-A}}{\sum_{i=0}^{N-1} \frac{A^i}{i!} + \frac{A^N}{N!} \cdot \frac{N}{N-A}} \quad t_c = \frac{t_{os} \cdot E_C}{N - A}$$

E_C — probability an arriving call must **wait** · t_{os} — mean holding time [s] · t_c — mean wait, **averaged over all calls**

Assumes blocked calls **stay in the system** until served, with unlimited waiting room (M/M/N). erlang.com/calculator/erlc

Erlang C — worked example

Same $N = 3$, $A = 1.5$ Erl scenario, now with unlimited waiting (M/M/3):

$$\frac{A^N}{N!} \cdot \frac{N}{N - A} = 0.5625 \cdot \frac{3}{1.5} = 1.125$$

$$E_C = \frac{1.125}{(1 + 1.5 + 1.125) + 1.125} = \frac{1.125}{4.75}$$

$$E_C \approx 23.7\%$$

with mean holding time $t_{os} = 3$ min:

$$t_c = \frac{3 \cdot 0.237}{1.5} \approx 0.47 \text{ min} \approx 28 \text{ s}$$

The Engset model

$$E_N(\alpha) = \frac{\binom{s-1}{N} \alpha^N}{\sum_{i=0}^N \binom{s-1}{i} \alpha^i}$$

s — number of traffic sources (finite population) · N — number of channels · α — offered traffic **per idle source**

Erlang B assumes an **infinite** population, so a busy source never "uses up" a potential arrival. With only s sources, a source already in a call **cannot** also be generating a new one — Engset accounts for this, and is more accurate for **low-traffic** scenarios with few sources.

Erlang B is the $s \rightarrow \infty$ limit of Engset. erlang.com/calculator/engset

Checkpoint — loss or wait?

A call centre wants to know whether to add a third agent or upgrade its phone system to put overflow calls on hold. Which formula answers which question — Erlang B or Erlang C?

Predict first · discuss with a neighbour · then reveal

Both are legitimate designs, computing different things.

Erlang B (no hold queue): with 2 agents, what fraction of callers gets a busy signal? **Erlang C** (calls held): with 2 agents and a hold queue, what fraction of callers wait, and how long on average? "Add a third agent" changes N in either formula; "put callers on hold" is the choice between the two models, not a formula parameter.

Notation cheat-sheet

Symbol	Meaning	Introduced
λ	Arrival rate [s^{-1}]	Part 1
μ	Service rate [s^{-1}]	Part 1
ρ	Utilization = λ/μ (M/M/1)	Part 3
L, L_q	Mean number in system / in queue	Part 3
W, W_q	Mean time in system / in queue	Part 3

Symbol	Meaning	Introduced
A	Offered traffic [Erl] (= $N\rho$ for N servers)	Part 4
N	Number of servers / channels	Parts 1, 4
E_B, E_C	Erlang B blocking / Erlang C waiting probability	Part 4
s	Number of traffic sources (Engset)	Part 4
α	Offered traffic per idle source (Engset)	Part 4

A and ρ both mean "offered traffic" — ρ is the M/M/1 name for it, A is the general telephony name used once there may be several servers.

Common mix-ups

$$\rho = \lambda/\mu \text{ vs. } \rho = A/k$$

The M/M/1 formulas for L , W , L_q , W_q assume **one** server. With k servers, utilization is A/k , and the M/M/1 occupancy/delay formulas do not apply directly — see Extras.

Erlang B vs. Erlang C

Same-looking formulas, different systems: B assumes **no** waiting room (blocked = lost); C assumes **unlimited** waiting room (blocked = delayed). Check which one a problem describes before reaching for either.

Convert units before you add

Little's law worked example mixed ms, s, and min on purpose — mixing units silently is the single most common arithmetic error with these formulas.

λ is the *admitted* rate

In Little's law and in $\rho = \lambda/\mu$, λ counts requests that actually **enter** the system. Blocked or lost traffic does not count toward it.

Summary — six formulas to keep

Little's law — any stable system

$$L = \lambda W$$

M/M/1 utilization

$$\rho = \lambda / \mu$$

M/M/1 occupancy and delay

$$L = \frac{\rho}{1 - \rho} \quad W = \frac{1}{\mu - \lambda}$$

Poisson counting

$$P(N=k) = \frac{(\lambda t)^k}{k!} e^{-\lambda t}$$

Erlang B — loss system

$$E_B(A, N) = \frac{A^N / N!}{\sum_{i=0}^N A^i / i!}$$

A steady state needs $\rho < 1$ (M/M/1) or $A < N$ (Erlang C). Kendall's notation says which model applies before any formula is chosen.

Exit questions — use the ideas

1. In Kendall's notation, what changes between M/M/1 and M/D/1, and which formulas from this lecture still apply?
2. Why does a memoryless service-time model make the M/M/1 formulas tractable?
3. Does doubling the arrival rate λ (at fixed μ) double the mean delay W ?
4. A trunk group blocks too many calls. Name two independent ways to reduce E_B .

Explain the assumptions, not just the formula.

Exercise 02 — what you will do

1. **Generate random samples** — uniform, exponential, and normal distributions; compare histograms with theory.
2. **Source and sink** — one `PacketSource`, then several, feeding a `PacketSink` directly.
3. **Source, switch, sink** — a buffered `SwitchPort`; record dropped packets as rate or buffer shrink.
4. **Network tap and M/M/1** — measure occupancy and delay against the analytical formulas, and check Little's law.

Then the README questions:

- a payphone under Erlang C — is a second line needed?
- λ, μ, ρ , and W vs W_q in Step 4
- why delay explodes as $\rho \rightarrow 1$
- which sampled distribution is memoryless, and why it matters

```
1  ssh <lab-server>
2  cd qos-02 && uv sync
3  uv run jupyter lab --ip 0.0.0.0
```

References — foundations

- D. G. Kendall, "Stochastic Processes Occurring in the Theory of Queues and their Analysis by the Method of the Embedded Markov Chain," *Annals of Mathematical Statistics*, 24(3), 338–354, 1953.
- J. D. C. Little, "A Proof for the Queuing Formula: $L = \lambda W$," *Operations Research*, 9(3), 383–387, 1961.
- A. K. Erlang, "The Theory of Probabilities and Telephone Conversations," 1909.
- A. K. Erlang, "Solution of some Problems in the Theory of Probabilities of Significance in Automatic Telephone Exchanges," 1917.
- T. O. Engset, "On the Calculation of Switches in an Automatic Telephone System," manuscript 1918 (English translation, *Teletronikk*, 1998).
- L. Kleinrock, *Queueing Systems, Volume I: Theory*. Wiley, 1975.

References — explanations and applications

- D. Gross, J. F. Shortle, J. M. Thompson, and C. M. Harris, *Fundamentals of Queueing Theory*, 4th ed. Wiley, 2008.
- Erlang traffic calculators: [Erlang B](#) · [Erlang C](#) · [Engset](#).
- G. Bernstein, "Discrete Event Simulation in Python," Grotto Networking — origin of the SimPy components used in Exercise 02.
- SimPy documentation, simpy.readthedocs.io.

Thank you for your attention

Jan Rozhon, Miroslav Vozňák

+420 596 995 900 • jan.rozhon@vsb.cz

www.vsb.cz

Extras

M/M/k, simulation components, and the erlang formulas' history

Extras — from one server to many: M/M/k

With k identical servers instead of one, offered traffic $A = \lambda/\mu$ spreads across all of them:

$$\rho = \frac{A}{k}$$

The M/M/1 formulas for L , W , L_q , W_q from Part 3 **do not apply directly** — M/M/k has its own (more involved) steady-state formulas, related to the Erlang C expression from Part 4.

A common mix-up: doubling the number of servers at fixed total offered traffic is **not** the same as doubling μ for one server — the two servers can each be idle while the other queues, unlike a single faster server.

M/M/k with **no** waiting room is exactly the Erlang B system (M/M/k/k); M/M/k with unlimited waiting room is exactly the Erlang C system.

Extras — the simulation building blocks

Component	Role
PacketSource	Generates packets with specified sizes and inter-arrival times
SwitchPort	Buffers waiting packets and transmits one packet at a fixed bit rate
Switch	Groups several ports
PacketSink	Records the elapsed time from packet creation to reception
NetworkTap	Samples system occupancy in packets, including a packet in service
PacketFork	Sends each packet to one of several destinations with given probabilities

Components connect through a `destination` attribute; sizes and intervals may be a number or a zero-argument callable, e.g. `partial(rng.exponential, 2)`.

Extras — Erlang's and Engset's papers

Year	Author	Contribution
1909	A. K. Erlang	<i>The Theory of Probabilities and Telephone Conversations</i> — proves the Poisson distribution applies to random telephone traffic
1917	A. K. Erlang	<i>Solution of some Problems...</i> — the classic loss (Erlang B) and waiting-time (Erlang C) formulas
1918	T. O. Engset	Finite-source loss formula, independently derived; published in English only in 1998
1953	D. G. Kendall	The A/B/C queueing notation used throughout this lecture

Erlang worked at the Copenhagen Telephone Company; the erlang (Erl) traffic unit is named for him in 1946 by the CCIF (a predecessor of today's ITU-T).

Extras — computing Erlang B by hand

$$E_B(0, A) = 1, \quad E_B(N, A) = \frac{A \cdot E_B(N-1, A)}{N + A \cdot E_B(N-1, A)}$$

Equivalent to the closed form on the earlier slide, but computed by **one multiplication and one division per step** instead of two large factorial sums — no term ever grows past 1.

$N = 3, A = 1.5$: $E_B(0)=1 \rightarrow E_B(1)=0.6 \rightarrow E_B(2)\approx 0.310 \rightarrow E_B(3)\approx 0.134$ — matches the worked example.

Extras — Little's law, decomposed

Subtracting the queue-only form of Little's law from the system-wide form:

$$L - L_q = \lambda(W - W_q) = \lambda \cdot \frac{1}{\mu} = \rho$$

The gap between system and queue occupancy is exactly ρ : the mean number of requests **in service** at any instant — the algebraic step behind the delay-decomposition figure shown in Part 3.